

Архиватор на базе алгоритма Хаффмана

Анпилогов Дарий Михайлович

Группа: 25.Б41-мм

Кафедра: *системного программирования*

Научный руководитель: *Литвинов Юрий Викторович*

Номер семестра практики: 2

1. Введение

Написание архиватора, основанного на базе алгоритма Хаффмана, — это учебная задача, направленная на получение опыта написания практической работы, изучение стандартных требований, процессов, а также способов достижения поставленных в рамках работы целей. Несмотря на наличие подобных решений, понимание проблем и требований при их разработке является важной частью обучения.

2. Постановка задачи

Целью работы является реализация консольного приложения, выполняющего в зависимости от параметров командной строки сжатие или разжатие файла по алгоритму Хаффмана.

Для достижения цели требуется решить следующие задачи:

1. Выполнить обзор существующих решений.
2. Выбрать инструменты для реализации.
3. Спроектировать решение.
4. Реализовать решение.
5. Выполнить экспериментальное исследование.

3. Обзор

Целью обзора является изучение алгоритма Хаффмана, его применимости, а также существующих подходов к его реализации для выбора архитектурного решения его реализации.

Алгоритм Хаффмана был предложен Дэвидом Хаффманом в 1952 году [1] и предназначен для сжатия данных без потерь. Алгоритм строит префиксный код переменной длины на основе частот появления символов во входных данных. Более часто встречающимся символам назначаются более короткие коды, а редким — более длинные. Для заданного набора частот алгоритм обеспечивает минимальную среднюю длину кодового слова среди всех префиксных кодов.

Классический алгоритм Хаффмана требует два прохода по исходным данным. Первый для подсчёта встречаемости символов, второй для кодирования.

Существуют различные вариации алгоритма Хаффмана, например Адаптивный алгоритм Хаффмана [2], позволяющий сжать данные за один проход, N-арный алгоритм Хаффмана (n-ary Huffman coding) [1], использующий алфавит размера N (в отличие от классического при $N = 2$), и множество других.

Алгоритм Хаффмана применяется как самостоятельный метод сжатия, а также используется в составе более сложных алгоритмов и форматов хранения данных. В частности, кодирование Хаффмана входит в состав алгоритма DEFLATE, реализованного в библиотеках zlib, 7-Zip, применяемого в форматах ZIP, gzip и PNG [3].

При реализации архиватора необходимо решить две основные задачи: построение дерева Хаффмана и сохранение информации, необходимой для восстановления кодов при распаковке. Для построения дерева в большинстве реализаций используется очередь с приоритетами, позволяющая эффективно выбирать узлы с минимальными частотами. В данной работе используется контейнер `std::priority_queue`, обеспечивающий построение дерева за $O(n \log n)$, где n — количество различных символов входных данных.

Для восстановления кодов при распаковке распространены различные подходы: сохранение таблицы частот символов с последующим построением дерева Хаффмана, хранение полной структуры дерева, сохранение обычных кодов Хаффмана в виде таблицы <символ><длина><код>, а также использование канонических кодов Хаффмана, при котором хранятся только длины кодов [1].

В данной работе выбран подход с хранением обычных кодов Хаффмана: для каждого символа сохраняется его значение, длина кода и битовое представление. По сравнению с хранением таблицы частот такой метод уменьшает объём служебной информации в архиве и упрощает процедуру декодирования, поскольку коды могут быть использованы напрямую без дополнительного построения дерева. Хотя канонические коды требуют ещё меньше места для хранения, явное хранение битовых представлений кодов упрощает реализацию алгоритма и незначительно увеличивает размер заголовка архива.

Таким образом, алгоритм Хаффмана является подходящей основой для реализации учебного архиватора благодаря сравнительно простой реализации и использованию эффективных структур данных, а также позволяет изучить работу с двоичными данными, файловым вводом-выводом, тестированием программного обеспечения и процессом разработки законченного программного решения.

4. Описание предлагаемого решения

Для реализации проекта были выбраны:

- Язык программирования: C++17. Причина — скорость и простота работы с потоками данных и битовыми операциями.
- Система сборки: CMake v3.14. Причина — простота сборки, кроссплатформенность.
- Тестирование: GoogleTest v1.14.0. Причина — стандарт индустрии, функциональность.
- Документация: Doxygen v1.9.8. Причина — стандарт индустрии, автоматизация генерации.
- CI: GitHub Actions. Причина — простота интеграции с платформой распространения проекта. Используется для запуска тестов и проверки программы с использованием clang-format, AddressSanitizer и UndefinedBehaviorSanitizer.

Реализация разделена на две части:

- Библиотека, реализующая функции архивации и разархивации.
- Консольное приложение, использующее библиотеку и предоставляющее пользователю интерфейс для использования функций.

Основными компонентами библиотеки являются:

- BitReader — чтение отдельных битов из входного потока с использованием внутреннего буфера;
- BitWriter — запись отдельных битов в выходной поток;
- Bits — контейнер для хранения битовых последовательностей;
- CodesTable — таблица кодов Хаффмана для всех возможных значений байта;
- CountTable — таблица частот встречаемости байтов (количества вхождений каждого байта);
- archive() — реализация алгоритма сжатия;
- unarchive() — реализация алгоритма разжатия.

Для построения кодов Хаффмана используется очередь с приоритетами `std::priority_queue`. Дерево в явном виде не строится. На каждой итерации алгоритма склеиваются два набора байтов с минимальными суммарными вхождениями в файл. В конец к кодам байтов добавляется по одному биту: для байтов из меньшего набора — 0, из большего — 1. В конце алгоритма все коды переворачиваются. Для разархивации явным образом строится дерево по таблице кодов.

Архив содержит размер таблицы кодов, описание таблицы кодов (количество используемых кодов, тройки <символ>, <длина кода>, <код>), размер сжатых данных в битах и сжатые данные. Не содержащиеся в исходном файле коды не хранятся в таблице кодов. Такой подход позволяет восстановить таблицу кодов без необходимости хранить полную таблицу частот.

Поддерживаются два режима работы архиватора. В режиме `twice` входной файл читается дважды: первый проход используется для подсчёта частот, второй — для кодирования данных. Данный режим позволяет обрабатывать файлы произвольного размера без полного размещения их содержимого в памяти. В режиме `ram` файл полностью загружается в оперативную память, что уменьшает количество операций ввода-вывода и позволяет работать с потоками, не поддерживающими повторное чтение.

Приложение поддерживает обработку ошибочных ситуаций: неверные аргументы командной строки, отсутствие входного файла, ошибки чтения и записи, а также повреждённые архивы. Для завершения работы используются различные коды возврата: 0 при успешном выполнении, 1 при ошибке параметров командной строки и -1 при прочих ошибках.

Для релизной версии дополнительно используются межпроцедурная оптимизация и оптимизации компилятора. Проект распространяется по лицензии MIT.

Реализовано как модульное, так и интеграционное тестирование — всего 47 тестов.

5. Эксперименты

Для проведения экспериментального исследования были выбраны следующие инструменты:

- Язык программирования: Python 3.13. Причина — простота кода, совместимость с различными типами данных.
- Менеджер зависимостей: uv. Причина — возможность явно указать используемые библиотеки, простой запуск.
- Формат сохранения данных: CSV. Причина — простота чтения и записи данных.
- Библиотека построения графиков: matplotlib. Причина — стандарт для отрисовки графиков, простота использования, распространённость.

- Библиотека для HTTP-запросов: requests. Причина — стандарт для http запросов в Python, простота использования, распространённость.

Для исследования производительности был разработан набор автоматизированных экспериментов. Измерения проводились на компьютере с процессором Intel Core i7-9750H, 16 ГиБ оперативной памяти под управлением операционной системы openSUSE Tumbleweed.

В качестве тестовых данных использовались файлы с частотами встречаемости символов, аналогичным следующим расширениям файлов: txt, csv, json, bmp, jpeg, png, wav, mp3, mp4, zip, 7z, exe. А также искусственно созданные файлы с содержимым, повторяющимся каждые 1, 2, 4, 8, 16, 32 байта и случайными данными. Файлы с данными для тестирования получены с сайтов samplefile.com, filesamples.com, www.gutenberg.org и github.com.

Размер файлов изменялся от 1 байта до 128 МиБ по степеням двойки. Для каждого измерения выполнялось 10 независимых запусков. Рассчитывались математическое ожидание и среднеквадратичное отклонение времени выполнения, а также скорости сжатия и разжатия.

Результаты показали, что коэффициент сжатия существенно зависит от энтропии входных данных. Для периодических последовательностей коэффициент сжатия стремится к значению 0.125, что соответствует восьмикратному уменьшению размера файла. Для данных с высокой энтропией эффективность сжатия снижается, а в отдельных случаях размер архива может превышать размер исходного файла из-за накладных расходов на хранение таблицы кодов.

Скорость сжатия для файлов размером от 10^6 байт колеблется от $0.2 \cdot 10^8$ до $1.5 \cdot 10^8$ байт в секунду. Для файлов меньшего размера скорость существенно ниже из-за значительных накладных расходов на построение кодов Хаффмана и выделение памяти под буферы ввода-вывода.

Скорость разжатия для файлов размером от 10^6 байт колеблется от $1.1 \cdot 10^7$ до $2.5 \cdot 10^7$ байт в секунду и в значительно слабее зависит от размера разжимаемого файла. Однако влияние выделения буферов остаётся.

Установлено, что рост энтропии данных приводит к почти линейному ухудшению коэффициента сжатия, а также к снижению скорости сжатия. Для разжатия только косвенно — в силу увеличенного размера разжимаемого файла. Полученные зависимости представлены графиками, построенными по результатам экспериментов.

Результаты экспериментов сведены в таблицу: https://github.com/GidRadium/huffman-archiver/blob/main/research/benchmark_results.csv.

Построенные графики доступны по ссылке: <https://github.com/GidRadium/huffman-archiver/blob/main/research/plots>.

6. Заключение

В ходе выполнения работы получены следующие результаты:

- выполнен обзор алгоритма Хаффмана и существующих подходов к построению архиваторов;
- разработано консольное приложение для сжатия и разжатия файлов по алгоритму Хаффмана;
- реализованы два режима архивирования, ориентированные на различные сценарии использования;
- разработан набор модульных и интеграционных тестов;
- проведено экспериментальное исследование эффективности и производительности реализации;
- подготовлена инфраструктура автоматической сборки, тестирования и генерации документации.

Репозиторий: <https://github.com/GidRadium/huffman-archiver>.

Техническая документация: <https://gidradium.github.io/huffman-archiver/>.

7. Литература

Список литературы

- [1] Wikipedia contributors. Huffman coding [Электронный ресурс]. URL: https://en.wikipedia.org/wiki/Huffman_coding (дата обращения: 16.06.2026).
- [2] Wikipedia contributors. Adaptive Huffman coding [Электронный ресурс]. URL: https://en.wikipedia.org/wiki/Adaptive_Huffman_coding (дата обращения: 16.06.2026).
- [3] Wikipedia contributors. Deflate [Электронный ресурс]. URL: <https://en.wikipedia.org/wiki/Deflate> (дата обращения: 16.06.2026).